

嵌入式计算机系统与实验 | 作业 9

Shuiyuan@Noroshi

1 UART 通信

回忆在 51 单片机时期的 UART, 我们知道 UART 通信大致思路是发送接收行为由 RX, TX 的状态控制, 传输时发送和接收方分别会进入发送中断和接收中断, 在发送结束之后解除. 在 MSP430 中主要的通讯模块是 USCI(Universal Serial Communication Interface), 其中的 USCI_Ax 也支持以 UART 的方式进行通讯.

MSP430 中选择 UART 的方式是清除 UCSYNC 位. 并且在初始化时, 为了防止出现不可预测的行为, 需要将 UCSWRST 位置为 1, 然后配置对应的端口引脚, 再清除 UCSWRST 位. 在输出状态下, CPU 会将数据写入 UCAXTXBUF, 并且通过 UART 的发送引脚逐位发送. 当发送缓存变空时, 其会发送一个 UCTXIFG 的发送中断, 在对应中断使能位打开时, 其会触发中断. 输入也是类似的原理, 其在接收引脚逐位接收数据, 并移入 UCAXRXBUF 寄存器中, 在完成接收之后, 其会设置一个接收中断标志位, 在对应的中断使能位打开时, 其就会触发输入中断, 通知 CPU 有新的数据可读.

1.1 简单示例

那么, 我们可以通过一个自己向自己发送数据的例子, 来展示 UART 通信的基本原理

```
#include <msp430f6638.h>
#include <stdint.h>
#include <stdio.h>
#include "dr_tft.h"

unsigned char flag0=0, flag1=0;
unsigned char send_data[]={ '0', '\0' };
unsigned char recv_data[]={ '0', '\0' };

void UART_RS232_Init(void);
void TimerA_Init(void);

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;

    UART_RS232_Init();
    initTFT();
    etft_AreaSet(0,0,319,239,0);
    TimerA_Init();
    _EINT();

    etft_DisplayString("Send Data: ",80,100,65535,0);
    etft_DisplayString("Recv Data: ",80,140,65535,0);

    while(1)
    {
        if(flag0)
```

```

    {
        flag0=0;
        UCA1TXBUF=send_data[0];
        etft_DisplayString(send_data,170,100,65535,0);
        send_data[0]++;
        if(send_data[0]>'9')
            send_data[0]='0';
    }

    if(flag1)
    {
        flag1=0;
        etft_DisplayString(recv_data,170,140,65535,0);
    }
}

#pragma vector=USCI_A1_VECTOR
__interrupt void USCI_A1_ISR(void)
{
    switch(__even_in_range(UCA1IV,4))
    {
        case 0:break;
        case 2:
            while(!(UCA1IFG&UCTXIFG));
            recv_data[0]=UCA1RXBUF;
            flag1=1;
            break;
        case 4:break;
        default:break;
    }
}

#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    static unsigned char i=0;
    i++;
    if(i>=20)
    {
        i=0;
        flag0=1;
    }
}

void UART_RS232_Init(void)
{
    P3DIR|=(1<<4)|(1<<5);
    P4DIR|=(1<<4)|(1<<5);
    P4OUT|=(1<<4);
    P4OUT&=~(1<<5);
    P3OUT|=(1<<5);
    P3OUT&=~(1<<4);
    P8SEL|=0x0c;
    UCA1CTL1|=UCSWRST;
    UCA1CTL1|=UCSSEL_1;
    UCA1BR0=0x03;
    UCA1BR1=0x00;
    UCA1MCTL=UCBRS_3+UCBRF_0;
    UCA1CTL1&=~UCSWRST;

```

```

    UCA1IE |= UCRXIE;
}

void TimerA_Init(void)
{
    TA0CTL |= MC_1 + TASSEL_2 + TACLK;
    TA0CCTL0 = CCIE;
    TA0CCR0 = 50000;
}
}

```

我们先不看这里与显示屏有关的内容, 重点关注 UART 的收发部分, 其通过 flag0 和 flag1 两个变量来判断是发送还是接收, 当 flag0 为 1 时, 其会进入发送状态, 发送完成后其会进入发送中断, 不过这里我们的中断程序忽略了发送中断的部分(即中断服务程序里的 case 4). 在其接收引脚开始接收数据后, 其接收的数据会按位移入 UCA1RXBUF 中, 在结束接收后, 其会进入接收中断(中断服务程序里的 case 2), 这部分我们是由对应的指令了, 具体就是当我们在接收中断的同时, 发送缓存区为空时, 其就会将接收的内容提供给 CPU(或者说从程序层面, 赋值给我们的变量)

根据定时器的设置, 其会每秒发送一次数据, 其发送给自己接收. 我们可以看看效果:



Figure 1: 如图, 我们可以发现 439 完成了自己发送自己接收的通讯

1.2 实现 430 与电脑之间的通讯

在完成上面那个简单的自己发给自己的任务之后, 我们可以来尝试一些更复杂和实际的通讯任务, 比如和计算机的通讯. 这里我们仍然使用 UART 和 RS232, 不过鉴于现在大部分个人 PC 都不会有 RS232 接口, 我们可以使用 USB 仿真的技术来实现通信.

先看代码, 不过这部分代码确实有点长, 我们就仅考虑部分

```

#pragma vector=USCI_A1_VECTOR
__interrupt void USCI_A1_ISR(void)
{
    static int recv_index=0;
    switch(__even_in_range(UCA1IV, 4)) {
        case 0: break;
        case 2:{
            char received = UCA1RXBUF;

```

```

char debug_buf[2];
debug_buf[0] = received;
debug_buf[1] = '\0';
etft_DisplayString(debug_buf, 250, 180, 65535, 0);

if(received == '\r' || received == '\n' || rcv_index >= RECV_BUF_SIZE - 1) {
    if(rcv_index > 0) {
        rcv_data[rcv_index] = '\0';
        flag1 = 1;
    }
    rcv_index = 0;
}
else if(received >= 32 && received <= 126) {
    rcv_data[rcv_index++] = received;
    rcv_data[rcv_index] = '\0';
    etft_DisplayString(rcv_data, 170, 140, 65535, 0);
}
break;
}
case 4: break;
default: break;
}
}
}

```

这部分就是电脑数据发送到 430 的部分，整体和之前的那个例子比较类似，不过这里我们考虑的是字符串而非字符了，因此我们还需要先维护一个临时字符串来存放我们的输入，并在检测到例如完成输出的信号或者缓冲区满的时候再一并将其输出在 tft 屏上。

然后我们再看看由 430 发送数据到电脑的部分

```

if(key_char == '#') {
    if(send_index > 0) {
        int j = 0;
        for(j = 0; j < send_index; j++) {
            while (!(UCA1IFG & UCTXIFG));
            UCA1TXBUF = send_buffer[j];
        }
        etft_DisplayString(send_buffer, 170, 100, 65535, 0);
        send_index = 0;
        memset(send_buffer, 0, SEND_BUF_SIZE);
        etft_DisplayString("          ", 170, 60, 65535, 0);
    }
}
}

```

这部分取自 ProcessKeyPress 函数，其和接收的步骤整体也是一样的，就是维护了一个缓冲字符串，然后把元素逐个写入发送寄存器，直到识别到终止符。

我们看看结果：

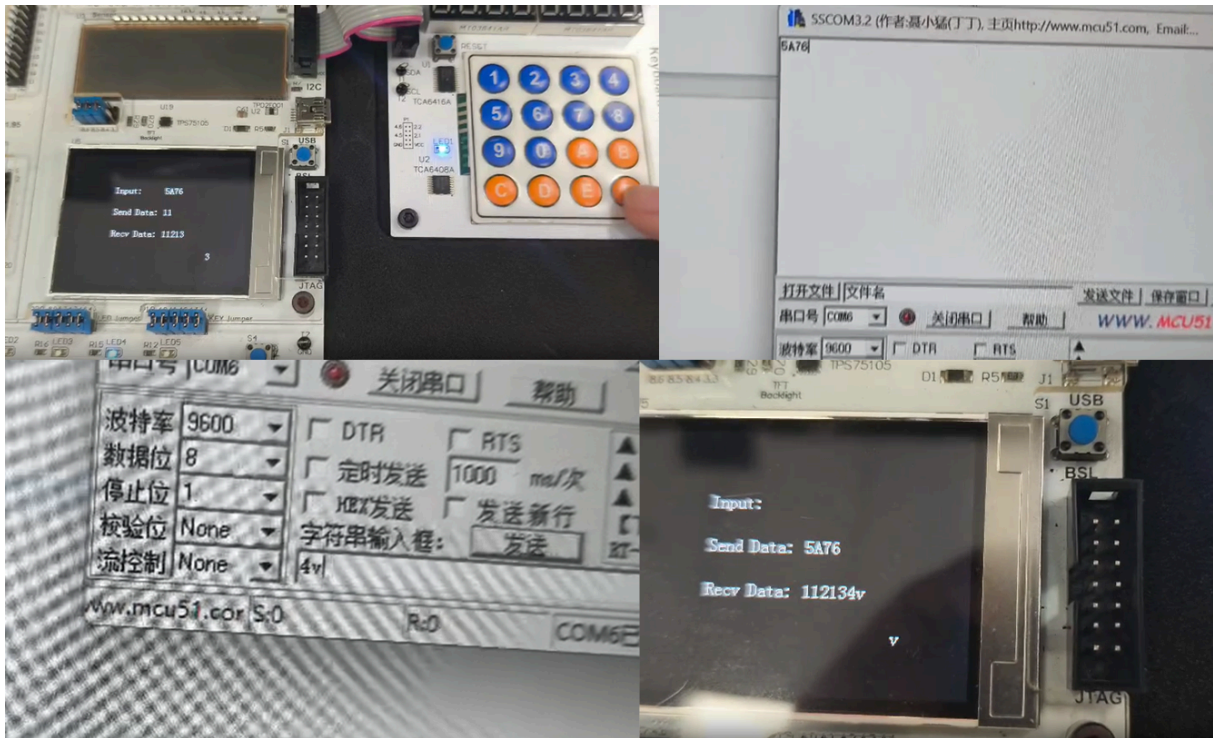


Figure 2: 左上, msp430 发送; 右上, 电脑接收; 左下, 电脑发送; 右下, msp430 接收. 具体演示视频见 lab9_2.mp4

2 SPI 与 tft-lcd

SPI(Serial Peripheral Interface)是另一种串行通讯接口, 其特点是在发送的时候会同步输出, 实现了全双工的效果. 其一般需要 4 根线: SCK 线, 用于主机发出自身的时钟信号, 作为从机的时钟输入. MISO/MOSI 线各一根, 分别传输主机接收, 从机输出(Master in, slave out)和主机输出, 从机接收(Master out, slave in)的数据流, 以及一根 NSS 线, 用于选择目标从机.

MSP430 的 USCI 模块中的 USCI_Bx 接口都支持 SPI 模式, 当 UCSYNC 位被设置, 并且通过 UCMODEx 位选择了 SPI 模式之后, USCI 模块就进入了 SPI 工作状态. 当 USCI 模式清除 UCSWRST 位使能之后, 其就会进入待收发状态. 作为主机, 其在写入 UCxTXBUF 寄存器后就会从待收发状态转为发送状态, 而对从机, 当片选信号为低电平有效时开始发送, 时钟信号由主机提供.

在我们的实验箱中, MSP430 通过 SPI 和一块 tft-lcd 屏幕连接. 我们可以通过这个外设来探究 SPI 通信基础.

我们完成的实验例程则是用其来显示汉字和图片, 对应的函数:

```
void etft_DisplayChinese16x16(uint16_t sx, uint16_t sy, const unsigned char *bitmap,
                             int char_count, uint16_t fRGB, uint16_t bRGB) {
    uint16_t cx, cy;
    int bytes_per_row = char_count * 2;
    unsigned char char_bitmap[32];
    int char_idx = 0;
    for(char_idx = 0; char_idx < char_count; char_idx++) {
        int row = 0;
        for(row = 0; row < 16; row++) {
            int src_pos = row * bytes_per_row + char_idx * 2;
            char_bitmap[row * 2] = bitmap[src_pos];
            char_bitmap[row * 2 + 1] = bitmap[src_pos + 1];
        }
        uint16_t char_x = sx + char_idx * 16;
    }
}
```

```

tft_SendCmd(TFTREG_WIN_MINX, char_x);
tft_SendCmd(TFTREG_WIN_MINY, sy);
tft_SendCmd(TFTREG_WIN_MAXX, char_x + 15);
tft_SendCmd(TFTREG_WIN_MAXY, sy + 15);
tft_SendCmd(TFTREG_RAM_XADDR, char_x);
tft_SendCmd(TFTREG_RAM_YADDR, sy);
tft_SendIndex(TFTREG_RAM_ACCESS);
for(cy = 0; cy < 16; cy++) {
    uint16_t line = (char_bitmap[cy * 2] << 8) | char_bitmap[cy * 2 + 1];
    for(cx = 0; cx < 16; cx++) {
        if(line & 0x8000) {
            tft_SendData(fRGB);
        } else {
            tft_SendData(bRGB);
        }
        line <<= 1;
    }
}
}
}

```

这部分函数的使用逻辑就是按照转换规则把中文字符转换为对应的数组之后, 从一角开始, 逐个像素逐个像素遍历判断其是否需要点亮.

对于图片也是类似的逻辑

```

void etft_DisplayImage(const uint8_t* image, uint16_t sx, uint16_t sy, uint16_t width,
uint16_t height)
{
    uint16_t i,j;
    uint32_t row_length = width * 3;
    if(row_length & 0x3)
    {
        row_length |= 0x03;
        row_length += 1;
    }
    const uint8_t *ptr = image + (height - 1) * row_length;
    tft_SendCmd(TFTREG_WIN_MINX, sx);
    tft_SendCmd(TFTREG_WIN_MINY, sy);
    tft_SendCmd(TFTREG_WIN_MAXX, sx + width - 1);
    tft_SendCmd(TFTREG_WIN_MAXY, sy + height - 1);

    tft_SendCmd(TFTREG_RAM_XADDR, sx);
    tft_SendCmd(TFTREG_RAM_YADDR, sy);

    tft_SendIndex(TFTREG_RAM_ACCESS);
    for(i=0;i<height;i++)
    {
        for(j=0;j<width;j++)
        {
            tft_SendData(etft_Color(ptr[2], ptr[1], ptr[0]));
            ptr += 3;
        }
        ptr -= width * 3 + row_length;
    }
}

```

不过这里我们需要注意的是在图片状态下其每个像素由三个数据控制(rgb), 因此应当注意遍历时步长为 3.

最后我们看看效果

